MILPBench: A Large-scale Benchmark Test Suite for Mixed Integer Linear Programming Problems

Huigen Ye
Tsinghua University
Beijing, China

Yaoyang Cheng
Hong Kong Baptist University
Hongkong, China

Hua Xu*
Tsinghua University
Beijing, China

Zhiguang Cao
Singapore Management University
Singapore, Singapore

Hanzhang Qin
National University of Singapore
Singapore, Singapore

Abstract

Mixed-integer linear programming (MILP) is a cornerstone of optimization with applications across numerous domains. However, the development and evaluation of MILP-solving algorithms are hindered by existing benchmark datasets, which are often limited in scale, lack diversity, and are poorly structured, making them inadequate for systematic testing across different solving approaches, especially for machine learning (ML)-based methods. To address these issues, we introduce MILPBench, a large-scale benchmark suite comprising 100,000 MILP instances organized into 60 well-categorized classes. Using structural properties and embedding similarity metrics, we developed a novel classification framework to ensure both intra-class homogeneity and inter-class diversity. In addition to the dataset, MILPBench includes a comprehensive baseline library featuring 15 mainstream solving methods, spanning traditional solvers, heuristic algorithms, and ML-based approaches. This design enables rigorous and standardized evaluation of MILP-solving algorithms under diverse conditions. Extensive benchmarking demonstrates the utility of MILPBench as a scalable and versatile testbed for advancing MILP research, fostering innovation in solver development, and bridging the gap between optimization and machine learning.

CCS Concepts

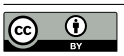
• **Theory of computation → Mixed discrete-continuous optimization**; *Integer programming*.

Keywords

Benchmark Dataset, Mixed Integer Linear Programming, Machine Learning, Branch-and-bound, Heuristic

ACM Reference Format:
Huigen Ye, Yaoyang Cheng, Hua Xu, Zhiguang Cao, and Hanzhang Qin. 2025. MILPBench: A Large-scale Benchmark Test Suite for Mixed Integer Linear Programming Problems. In *Genetic and Evolutionary Computation Conference (GECCO '25), July 14–18, 2025, Malaga, Spain.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3712256.3726324

*Hua Xu is the corresponding author (xuhua@tsinghua.edu.cn).



This work is licensed under a Creative Commons Attribution 4.0 International License.
GECCO '25, July 14–18, 2025, Malaga, Spain
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1465-8/2025/07
https://doi.org/10.1145/3712256.3726324

1 Introduction

Mixed-integer linear programming (MILP) is a cornerstone of optimization, widely applied in fields such as routing, scheduling, network design, and resource allocation [6, 28, 31, 33, 39, 41, 44, 47, 49, 51, 58, 73]. MILP involves optimizing a linear objective function under constraints with both integer and continuous variables [65, 66], providing solutions to complex combinatorial problems. Given its ubiquity and importance, advancing algorithmic efficiency for solving MILPs is critical for addressing real-world challenges.

Traditional MILP solvers can broadly be categorized into **exact algorithms** and **heuristic approaches** [72]. Exact methods, such as branch-and-bound [5, 38, 71], include techniques like pseudo-cost branching [12, 48], strong branching [4, 24], and hybrid branching [3, 64]. Heuristic methods, on the other hand, focus on approximate solutions, leveraging techniques such as feasibility pumps [13, 27], evolutionary algorithms [52, 59], and large neighborhood search [40, 55]. While these methods are highly effective, they often struggle in scenarios requiring the repeated solution of homogeneous MILP instances (i.e., problems with similar combinatorial structures). These scenarios expose traditional solvers to the challenge of **cold starts**, as they are unable to exploit accumulated solving knowledge to accelerate subsequent computations [54, 72].

In recent years, researchers have explored **machine learning (ML)** to improve MILP-solving methods. Inspired by the bipartite graph representation of MILP problems [30], neural networks— particularly graph neural networks (GNNs) [60]—have been employed to enhance branch-and-bound decisions [17, 34, 35], neighborhood selection [61, 62, 67], and heuristic initialization [25, 69, 70]. These methods promise significant speedups and improved scalability. However, despite their potential, progress has been hindered by the lack of **large-scale, systematically categorized benchmark datasets**. Existing datasets, such as MIPLIB [32], are limited in size, diversity, and structural organization. Moreover, none of these datasets provide a **baseline library** for standardized performance comparison across traditional solvers, heuristic algorithms, and ML-based approaches. This gap has slowed MILP solver development and impeded rigorous comparisons under diverse conditions.

To address these challenges, we introduce **MILPBench**, the largest MILP benchmark dataset to date, comprising 100,000 instances across 60 systematically categorized classes. MILPBench provides a diverse and comprehensive testbed for evaluating MILP-solving algorithms. By leveraging structural similarity metrics and

GNN-based embeddings, we ensure that MILPBench achieves intra-class homogeneity and inter-class diversity, enabling robust evaluations. In addition, MILPBench includes a baseline library featuring 15 mainstream solving methods, spanning traditional solvers, heuristic algorithms, and ML-based approaches, making it the first dataset to offer such a complete evaluation framework. Experimental results demonstrate that MILPBench enables detailed performance comparisons and reveals inconsistencies in some algorithm claims, further underscoring its value as a standardized and scalable testbed for MILP research.

In summary, MILPBench offers the following contributions:

(1) **Largest-Scale Dataset**: MILPBench is the largest MILP benchmark dataset to date, comprising 100,000 instances across 60 heterogeneous classes.
(2) **Comprehensive Baseline Library**: MILPBench provides a standardized evaluation framework with 15 mainstream solving methods, supporting traditional solvers, heuristic algorithms, and ML-based approaches alike.
(3) **Scalability and Adaptability**: MILPBench offers a diverse test suite with plans for regular updates, ensuring its relevance for future research and applications.

By addressing the limitations of existing datasets, MILPBench establishes a new standard for MILP benchmarking, facilitating algorithm development and fostering innovation across both traditional and ML-based approaches. Our MILPBench is open-source and accessible at: https://github.com/thuiar/MILPBench.

2 Related Work

2.1 Mixed Integer Linear Programming Problem

Mixed Integer Linear Programming (MILP) is a foundational class of combinatorial optimization problems with extensive applications in diverse fields, including logistics, robotics, and operations research. Formally, a MILP problem can be defined as follows [65, 66]:

$$\min_x \ c^T x, \text{subject to } Ax \le b, l \le x \le u, x_i \in \mathbb{Z}, i \in \mathbb{I}, \tag{1}$$

where x represents the decision variables, with dimension denoted by $n \in \mathbb{Z}$, and $l, u, c \in \mathbb{R}^n$ correspond to the lower bounds, upper bounds, and coefficient values of the variables, respectively. The matrix $A \in \mathbb{R}^{m \times n}$ and the vector $b \in \mathbb{R}^m$ define the linear constraints of the problem. The set $\mathbb{I} \subseteq \{1, 2, \ldots, n\}$ denotes the indices of variables that are constrained to be integers.

2.2 Existing Methods for Solving MILP

Solving MILP problems effectively has been a long-standing challenge, leading to the development of various algorithmic approaches. Traditional methods, including branch-and-bound, branch-and-cut, and cutting-plane algorithms, guarantee optimal solutions but often suffer from scalability issues when dealing with large problem instances. In contrast, heuristic approaches, such as genetic algorithms, simulated annealing, and large neighborhood search, offer faster solutions by sacrificing optimality. The availability of well-organized test datasets is crucial for understanding the performance of these methods across diverse scenarios, enabling researchers to identify their strengths and weaknesses and make informed choices for specific problem settings.

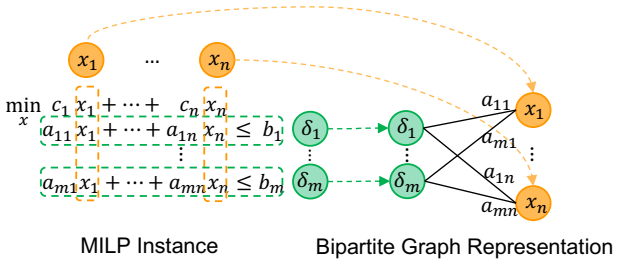


Figure 1: Convert MILP into bipartite graph.

In recent years, machine learning (ML) has emerged as a promising tool for solving MILP problems, with most frameworks relying on Graph Neural Networks (GNNs) to exploit the structural properties of MILP. As shown in Figure 1, Gasse et al. proposed a bipartite graph representation of MILP problems [30], which achieves a lossless transformation of MILP instances into graph structures. Such graph representations are well-suited for neural embedding networks [54], enabling ML-based approaches to learn optimization strategies directly from problem instances. These methods have demonstrated potential in bridging the gap between traditional solvers and modern data-driven techniques, offering new opportunities for improving solver efficiency and adaptability.

However, the lack of well-structured and carefully categorized datasets poses a significant challenge for both traditional and ML-based approaches. Homogeneous datasets are particularly important for ML-based methods, as heterogeneous problems can lead to poor model generalization. Previous studies [19, 56] have shown that GNNs often suffer from over-smoothing and limited expressiveness, which hinder their ability to handle diverse problem distributions. Furthermore, the generalization of large models with strong out-of-distribution capabilities also benefits from fine-grained category labels, which can help capture the structural differences between problem types during pretraining. These findings highlight the necessity of datasets with intra-class homogeneity and inter-class diversity to support both current ML techniques and the future development of large-scale optimization models.

2.3 Related Benchmark Datasets

The development and evaluation of mixed-integer linear programming (MILP)-solving methods, particularly ML-based approaches, are significantly hindered by the limitations of existing datasets. Widely used datasets such as MIPLIB [46] and Coral [21] provide general MILP instances but lack the structure and diversity needed for systematic evaluation. These datasets often fail to cover a wide range of problem scales and structural variations, making them inadequate for testing solvers designed for large-scale or domain-specific applications. Moreover, they are not tailored for ML research, as they lack fine-grained categorizations critical for training and evaluating ML models. Distributional MIPLIB [42] has made some attempts to address these issues by introducing MILP distributions from 13 domains, classified into different hardness levels. However, similar to other datasets, it is limited by its small number of problem categories (13 classes), small-scale problem instances (variables and constraints at most in the tens of thousands), reliance

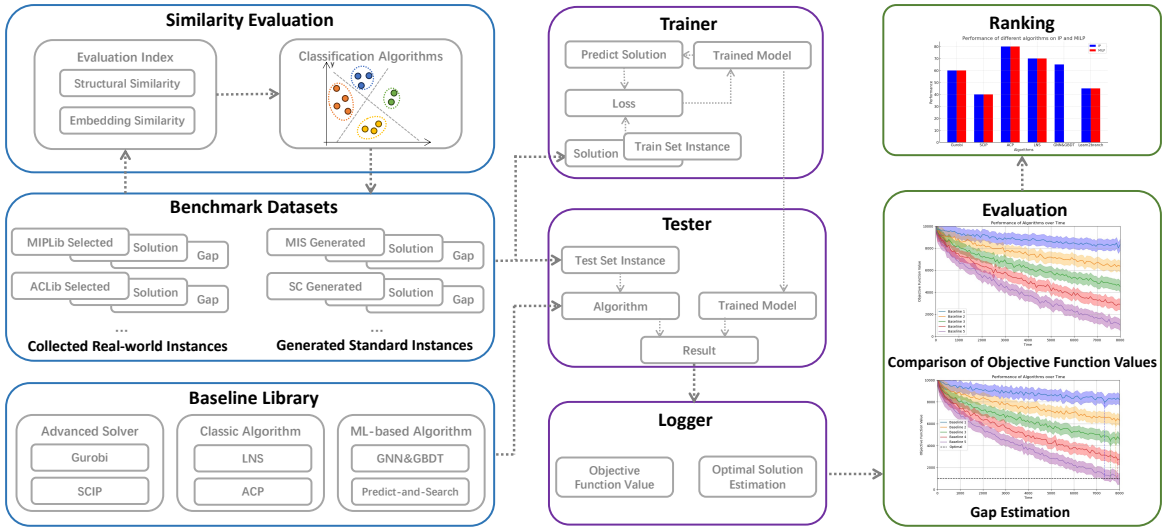


Figure 2: An overview of *MILPBench*. The *Benchmark Datasets* component organizes collected MILP instances into 60 well-categorized classes, guided by the *Similarity Evaluation* component, which uses structural and embedding-based metrics to ensure dataset quality. The *Baseline Library* component enables standardized training, testing, and logging of solving algorithms, with performance evaluated based on objective function values, gap metrics, and runtime efficiency. Results are aggregated into a leaderboard to facilitate systematic comparisons across algorithms.

on synthetic problem generation, and lack of real-world problem curation or an accompanying baseline library.

To address these limitations, we introduce MILPBench, a large-scale benchmark tailored to the needs of both traditional and ML-based MILP-solving methods. Unlike existing datasets, MILPBench offers extensive structural diversity, fine-grained categorizations, and realistic problem scales, paving the way for more rigorous and generalizable evaluation.

3 Proposed MILPBench

We propose MILPBench, a large-scale benchmark suite designed to address the limitations of existing datasets and enable systematic evaluation of MILP-solving algorithms across diverse conditions. MILPBench provides a scalable and versatile platform for evaluating a wide range of solving techniques, including traditional solvers, heuristic algorithms, and ML-based approaches. By offering a comprehensive dataset and baseline library, MILPBench bridges the gap between academic benchmarks and real-world applications, supporting the advancement of MILP research and fostering innovation in solver development.

As illustrated in Figure 2, MILPBench comprises three core components: Benchmark Datasets, Similarity Evaluation, and Baseline Library. The Benchmark Datasets include 100,000 MILP instances organized into 60 well-categorized classes, ensuring both intra-class homogeneity and inter-class diversity. This is achieved through a novel classification framework that integrates structural properties and embedding similarity metrics to analyze and categorize problem instances. The Similarity Evaluation component ensures that instances with low similarity scores are reclassified, maintaining the quality and consistency of the dataset. To enable rigorous and standardized testing, the Baseline Library provides 15 mainstream solving methods spanning traditional solvers, heuristic algorithms, and ML-based approaches. These methods are evaluated using a standardized "*Trainer-Tester-Logger*" pipeline, which measures solver performance based on objective function values, gap estimates, and runtime efficiency under fixed wall-clock times. Results are aggregated and ranked to create a comprehensive leaderboard, offering insights into the strengths and weaknesses of different solving techniques. This robust evaluation framework ensures that MILPBench serves as a scalable and practical testbed for advancing MILP research.

3.1 Benchmark Datasets

To enable the systematic evaluation of MILP-solving algorithms, we constructed a comprehensive benchmark dataset that consists of two main components: MILP Instances and their corresponding Reference Solutions and Gap Estimates. The dataset is designed to meet the diverse needs of solving techniques, ranging from traditional solvers to heuristic algorithms and machine learning (ML)-based approaches. By combining real-world problems and standardized mathematical formulations, the dataset ensures broad coverage across different problem types, scales, and complexities.

The MILP instances in our dataset originate from two primary sources. The first source includes real-world problems carefully curated from a wide range of open-source datasets and academic studies. These include mainstream datasets such as MIPLIB [46], AClib [43], Regions200 [50], MIRPLib [57], and COR@L [21]. Additionally, we incorporated problems from domain-specific studies, such as robustness verification of neural networks [54], lot-sizing polytopes [10], network design [8], combinatorial auctions [23],

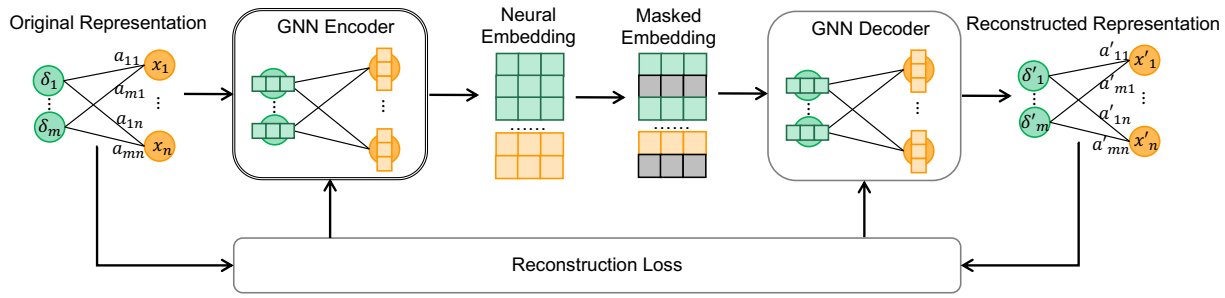


Figure 3: The overview of graph self-supervised learning. For the bipartite graph representation of MILP, we use the encoder-decoder structure for reconstruction, and the loss is computed by comparing the output with the features of the nodes in the original representation.

and 0-1 knapsack problems [11]. Competitions such as the ML4CO challenge at NeurIPS 2021 [29] and the Reoptimization Competition 2023 [15] were also valuable sources for real-world MILP problems. These curated instances reflect a wide variety of application domains, but many of them are small in scale and fail to represent the complexity of large-scale scenarios. Specific details about these datasets can be found in our online repository.

To address the limitations of small-scale real-world problems, the second source of our MILP instances consists of problems generated from standardized mathematical formulations. Specifically, we constructed instances based on nine canonical MILP problem classes, including Maximum Independent Set [63], Minimum Vertex Covering [26], Set Covering [16], Mixed Integer Knapsack Set [9], and Capacitated Facility Location [7]. For each problem type, we generated instances at three levels of difficulty—easy, medium, and hard—corresponding to tens of thousands, hundreds of thousands, and millions of decision variables, respectively. This approach ensures that the dataset includes ultra-large-scale problems, bridging the gap between academic benchmarks and real-world applications. Further details of the process are provided in our online repository.

Once the MILP instances were collected, we applied the Similarity Evaluation Metrics described in Section 3.2.1 to assess the homogeneity of the dataset. Instances with significant internal variability were re-screened and re-classified using the Classification Algorithm outlined in Section 3.2.2. This process ensures that the final dataset achieves a high degree of intra-class homogeneity and inter-class diversity, facilitating robust and systematic evaluation of solving techniques.

For each MILP instance, we also provided high-quality reference solutions and gap estimates to support benchmarking. To obtain these, we first employed the state-of-the-art solver Gurobi (version 11.0.1) to solve the problems for a fixed duration (e.g., 8 hours), during which most problems achieved optimal solutions. These solutions, along with the problem instances, were saved in pickle files for seamless integration into experiments. However, for larger-scale problems where Gurobi could not compute optimal solutions within the allocated time, we utilized an Adaptive Constraints Partition (ACP)-based strategy [68] to iteratively improve the solutions. Starting with the solution $\overline{x}$ and gap estimate $\overline{g}$ obtained from Gurobi, we computed the improved solution x^* and its corresponding gap g^* using the following formula:

$$g^* = \frac{x^* - (1 - \overline{g})\overline{x}}{x^*}. \tag{2}$$

This approach ensures that high-quality reference solutions are available even for large-scale problems. Detailed descriptions of the ACP-based strategy are presented in our online repository. Finally, the dataset was partitioned into training and testing subsets to support algorithm development and evaluation. Specific details of the partitioning scheme are provided in our online repository.

3.2 Similarity Evaluation

To ensure the quality and utility of MILPBench, we introduce a novel Similarity Evaluation framework that addresses the limitations of existing datasets. By integrating structural properties and embedding similarity metrics, this framework ensures that the 100,000 MILP instances in MILPBench are organized into 60 classes with both intra-class homogeneity and inter-class diversity. The framework consists of two key components: **Similarity Evaluation Metrics**, which assess the structural and embedding-based characteristics of MILP instances to quantify their similarity, and a **Classification Algorithm**, which leverages these metrics to systematically categorize and refine the dataset. This categorization process mitigates challenges such as over-smoothing and poor generalization in GNN-based methods [56], while also enhancing the applicability of traditional and heuristic solvers through a more systematic evaluation setup.

3.2.1 Similarity Evaluation Metrics. To quantify the similarity between MILP instances, we employ a two-pronged approach that combines structural embedding and neural embedding. These methods capture complementary aspects of MILP problem structures, ensuring a comprehensive evaluation of similarity.

Structural Embedding: Each MILP instance is represented as a 10-dimensional feature vector, summarizing key structural properties derived from its mathematical formulation and bipartite graph representation. Table 1 lists the metrics used, including features such as the fraction of non-zero entries in the coefficient matrix (`coef_dens`), the mean and standard deviation of variable and constraint vertex degrees (`var_degree_mean`, `cons_degree_std`), and the modularity of the graph. These metrics are selected to provide

Table 1: Structural Metrics for Similarity Evaluation.

Metric Name	Description
coef_dens	Fraction of non-zero entries in the coefficient matrix.
cons_degree_mean	Mean degree of constraint vertices in the bipartite graph.
cons_degree_std	Standard deviation of degrees of constraint vertices.
var_degree_mean	Mean degree of variable vertices in the bipartite graph.
var_degree_std	Standard deviation of degrees of variable vertices.
lhs_mean	Mean of non-zero entries in the coefficient matrix (LHS).
lhs_std	Standard deviation of non-zero entries in the coefficient matrix (LHS).
rhs_mean	Mean of right-hand-side (RHS) values in constraints.
rhs_std	Standard deviation of right-hand-side (RHS) values in constraints.
modularity	Modularity of the bipartite graph, measuring community structure.

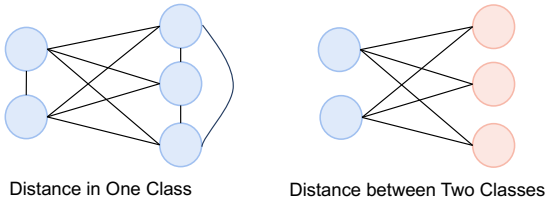


Figure 4: The Calculation of Similarity Score.

a holistic view of the MILP instance's structural characteristics. To evaluate the similarity of MILP instances based on these features, we compute the Jensen-Shannon (JS) divergence for each metric. Let JS_i denote the JS divergence for the ith metric. The similarity score for the ith metric is defined as:

$$\text{score}_i = \frac{\max(JS) - JS_i}{\max(JS) - \min(JS)}. \tag{3}$$

The overall structural similarity score is calculated as the average of all individual metric scores:

$$\text{score} = \frac{1}{10} \sum_{i=1}^{10} \text{score}_i. \tag{4}$$

This scoring system ensures that instances with similar structural properties are grouped together, facilitating systematic evaluation and classification.

Neural Embedding: To model intricate relationships such as local connectivity and coefficient interactions that are not fully captured by structural embedding, we adopt a graph self-supervised learning paradigm inspired by Graph Autoencoders [45]. MILP instances are represented as bipartite graphs and encoded using a Graph Convolutional Neural Network (GNN) encoder. As illustrated in Figure 3, the encoder maps the graph into an intermediate neural embedding, while the decoder reconstructs the original graph. The reconstruction loss, computed as the discrepancy between the reconstructed graph and the original input, ensures that the embeddings capture meaningful structural patterns. To further enhance the embeddings, we mask parts of the graph (both node features and edges) during training, forcing the model to predict the masked information. Formally, let $\mathcal{G} = (\mathcal{V}, X, E)$ represent a bipartite graph, where $\mathcal{V}$, X, and E are the nodes, node features, and edges, respectively. During masking, a subset of nodes $\mathcal{V}_{\text{mask}}$ and edges $\mathcal{E}_{\text{mask}}$ are selected with probabilities p_v and p_e, and their features are replaced with a special [mask] token. This process ensures that the learned embeddings encode both global and local structural features, making them robust for downstream similarity evaluations.

Similarity Measure: To evaluate the similarity of a set of MILP instances $\mathcal{I}$, we calculate the average pairwise Euclidean distance between their embeddings:

$$\text{Embedding Distance} = \frac{\sum_{\mathcal{I}_i, \mathcal{I}_j \in \mathcal{I}, i \neq j} \text{Distance}(\mathcal{I}_i, \mathcal{I}_j)}{|\mathcal{I}|(|\mathcal{I}| - 1)}, \tag{5}$$

where $\text{Distance}(\mathcal{I}_i, \mathcal{I}_j)$ denotes the Euclidean distance between the embeddings of instances $\mathcal{I}_i$ and $\mathcal{I}_j$. A lower embedding distance indicates higher similarity among instances, as illustrated in Figure 3. By integrating structural and neural embeddings, this hybrid approach provides a robust and scalable metric for evaluating MILP instance similarity.

3.2.2 Classification Algorithm. For datasets with low internal similarity, we propose a novel strategy to re-screen and re-classify the data, ensuring greater homogeneity within each subset. This process begins by applying the neural embedding method described in Section 3.2.1 to generate embeddings for all problem instances. These embeddings, which capture both global and local structural features, are then used as inputs to a spectral clustering algorithm. Spectral clustering constructs a similarity graph, where nodes represent problem instances and edge weights encode pairwise similarities. By analyzing the graph's structure through the eigenvectors of its Laplacian matrix, spectral clustering effectively identifies groups of instances with high internal similarity, even in cases where clusters are non-convex or non-linearly separable.

After the initial clustering, we evaluate the similarity metrics within each subset to assess their internal homogeneity. Subsets with sufficiently high similarity scores are retained, while those with lower scores are further subdivided using the same spectral clustering approach. This iterative refinement process, as illustrated in the right panel of Figure 4, ensures that the final dataset is well-structured, with clearly defined categories that exhibit both intra-class homogeneity and inter-class diversity. By leveraging this classification algorithm, we provide a robust foundation for systematic evaluation and training of solving algorithms, facilitating both research advancements and practical applications.

3.3 Baseline Library

To validate the effectiveness of MILPBench, we developed a comprehensive Baseline Library consisting of 15 mainstream solving methods. These methods span traditional solvers, heuristic algorithms, and ML-based approaches, providing a broad spectrum of techniques for rigorous benchmarking and comparison. This library ensures that MILPBench can evaluate solving techniques across diverse methodologies and application scenarios.

The first category in the Baseline Library includes state-of-the-art traditional solvers, such as SCIP (version 4.3.0) [2], Gurobi (version 11.0.1) [36] and CPLEX(version 22.1.1.0) [20]. These solvers represent the leading academic and commercial optimization tools and are widely used in both research and industrial applications. We implemented these solvers using their official APIs to solve specific MILP problems. The second category consists of heuristic algorithms, including Large Neighborhood Search (LNS) [40], General Large Neighborhood Search (GLNS) [61], Least Integeal

Table 2: The distance of graph structure and neural embedding in each heterogeneous classes. The blue background colour indicates classical MILP datasets.

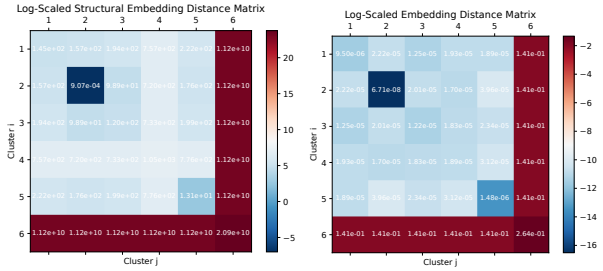
	Structure Distance	Embedding Distance
MIS_easy	0.011	1.50e-08
MVC_easy	0.010	1.24e-08
SC_easy	0.009	3.69e-08
Aclib	25.402	2.52e-06
Cut	7.122	1.47e-05
fc.data	5.867	9.68e-07
Hem_knapsack	0.566	8.75e-07
Hem_mis	0.356	4.29e-07
Hem_setcover	0.286	5.33e-07
Hem_corlat	5.956	3.34e-07
Hem_mik	226.520	9.54e-06
item_placement	0.037	7.03e-12
load_balance	0.436	6.97e-07
anonymous	222.061	8.99e-06
nn_verification	18.460	3.71e-06
vary_bounds_s1	0.001	7.08e-08
vary_bounds_s2	0.009	3.26e-08
vary_bounds_s3	0.009	1.90e-08
vary_matrix_s1	0.011	0.0
vary_matrix_rhs_bounds_s1	0.008	1.74e-08
vary_obj_s1	3.18e-06	8.39e-07
vary_obj_s2	1.22e-05	2.64e-07
vary_rhs_s1	0.007	9.83e-08
vary_rhs_s2	0.550	1.11e-05
vary_rhs_s4	0.601	1.47e-05
Transportation	148.368	1.10e-05
Coral	1056768.799	6.03e-03
ECOGCNN	468055.052	5.41e-02
MIPlib	7002615758.102	8.82e-02
Nexp	5511575859.716	2.48e-01

Heuristic (LIH) [53], Most Integeal Heuristic (MIH) [14], Relaxation Induced Neighborhood Search (RINS) [22] and Adaptive Constraint Partitioning (ACP) Optimization Framework [68]. These algorithms were reproduced based on their published pseudocode and adapted to work seamlessly with the MILP instances in our dataset. This category offers robust heuristic approaches that balance solution quality and computational efficiency. The third category comprises ML-based solving algorithms, which leverage advanced learning techniques to improve MILP-solving performance. These include Learn2Branch [30], Hybrid-Learn2Branch [34], GNN &GBDT-guided Optimization Framework [70], Neural Diving [54], Predict&Search Optimization Framework[37], and GNN-MILP [18]. These methods were retrained and tested using benchmark datasets, ensuring compatibility and fair evaluation.

Additionally, we experimented with other solving algorithms, such as Feasible Pump [27] and Simulated Annealing [1]. However, due to their inability to produce feasible solutions within reasonable computational limits for most problems, these methods were ultimately excluded from the final Baseline Library. Overall, the Baseline Library, combined with the Benchmark Datasets, provides a robust and standardized platform for evaluating and comparing the performance of diverse MILP-solving techniques.

4 Experiments

To validate the effectiveness of MILPBench, we conducted a series of experiments designed to highlight its advantages over traditional MILP datasets, such as MIPLib, Coral, and Nexp. These experiments systematically demonstrate MILPBench's ability to provide more homogeneous problem subsets, its robust classification algorithm, and its utility for benchmarking optimization algorithms. Specifically, we first compare the internal similarity of problem instances



(a) Structure distance matrix. (b) Embedding distance matrix.

Figure 5: Structure and embedding distance within classes.

in MILPBench against traditional datasets (Section 4.1), showcasing its superior ability to group structurally and semantically similar instances. Next, we analyze the distance matrix between problem categories (Section 4.2), validating the effectiveness of the classification method. Finally, we evaluate the performance of various benchmark algorithms on categorized problem subsets (Section 4.3), exposing potential weaknesses in state-of-the-art methods and demonstrating MILPBench's value in advancing the field.

4.1 Dataset Analysis

To validate the benefits of MILPBench over traditional MILP datasets such as MIPLib, Coral, and Nexp in evaluating ML-based optimization algorithms, we conducted a comparative analysis focusing on problem similarity. The experimental configurations are provided in our online repository for reproducibility.

The results, detailed in Table 2, reveal a striking contrast in problem similarity. For graph structural embedding, traditional datasets like MIPLib exhibited remarkably high variability, with pairwise distances between problem instances often exceeding hundreds of millions of times those observed in MILPBench. Similarly, the neural embedding distances in traditional datasets were found to be tens of thousands of times greater than those in MILPBench. These results demonstrate that MILPBench achieves significantly lower variability between instances, both in terms of structural and neural embeddings. This reduced variability directly translates to enhanced homogeneity within MILPBench. By providing problem instances that are more structurally and semantically similar, MILPBench enables more systematic evaluation of ML-based optimization algorithms. This is in stark contrast to traditional datasets, where the wide variability among instances often leads to inconsistent and less interpretable evaluation outcomes. MILPBench addresses this challenge by offering a well-curated dataset designed to promote consistency and comparability, ultimately advancing the development and benchmarking of optimization algorithms.

4.2 Dataset Reclassification

In some datasets within the collection, particularly those derived from open-source sources, problem instances generated from diverse scenarios are often intermingled. This results in highly complex distributions, making these datasets unsuitable for direct use as training data for ML-based optimization algorithms. To address

Table 3: Gap estimation of baselines. + represents the problem of scale being too large to accept the time to collect training samples. ! represents the problem of errors during band training. - represents MILP problems that GNN&GBDT cannot solve.

	Gurobi	SCIP	CPLEX	GLNS	ACP	RINS	Learn2branch	GNN&GBDT	Predict&Search	Time
MIS_hard	0.1714	53.4844	1.1265	0.1714	0.1184	0.2998	+	**0.1169**	+	4000s
MVC_hard	0.1310	91.8785	0.7098	0.1018	0.1077	0.1961	+	**0.0951**	+	4000s
SC_hard	0.9920	425323.8	+	0.9852	**0.9850**	0.9902	+	0.9887	+	4000s
MIPlib	**0.0000**	0.0587	**0.0000**	0.2157	0.0004	0.2331	0.3363	-	**0.0000**	150s
Coral	2.85e+04	2.857e+19	4.96e+05	3.05e+04	3.05e+04	3.52e+04	+	-	**2.33e+04**	4000s
Cut	0.1490	0.5387	**0.1046**	0.2744	0.1651	0.3781	0.5782	-	0.1568	4000s
ECOGCNN	**0.2512**	4.6056	0.2899	0.2730	0.2516	0.2814	+	-	**0.2512**	4000s
HEM_knapsack	**0.0000**	0.0000	0.0000	**0.0000**	**0.0000**	0.0117	**0.0000**	**0.0000**	**0.0000**	100s
HEM_corlat	**0.0000**	0.0000	0.0000	0.0129	**0.0000**	0.0141	!	-	**0.0000**	100s
HEM_mik	**0.0000**	0.0000	0.0000	0.0034	0.0146	0.0467	!	-	3.00e-15	100s
item_placement	**0.6481**	2.332e+07	0.9204	0.8431	0.8076	0.9971	4.17e+07	-	0.6595	4000s
load_balancing	0.0028	0.0273	**0.0003**	0.0227	0.0035	0.0616	+	-	0.0028	1000s
anonymous	0.3088	4.2244	0.3386	0.9256	0.5449	0.8876	+	-	**0.2909**	4000s
Nexp	0.0787	0.1514	0.0813	0.1095	0.0754	0.0989	0.1629	-	**0.0759**	4000s
Transportation	**0.1512**	0.2575	0.1646	0.2490	0.1767	0.1962	0.2725	-	0.1568	4000s
vary_bounds_s1	**0.0000**	0.0485	0.0001	0.3956	**0.0000**	0.4536	0.1518	-	0.0003	400s
vary_matrix_s1	**0.0000**	0.3796	0.0001	0.0008	0.0008	0.2443	0.4002	-	**0.0000**	100s
vary_obj_s1	**0.0000**	0.0031	0.0000	0.0019	**0.0000**	0.0012	0.0054	**0.0000**	**0.0000**	100s
vary_rhs_s1	0.0003	0.0364	0.0000	5.5134	0.2037	1.7870	+	-	**0.0000**	100s
Aclib	**0.0000**	0.0000	0.0001	0.0006	0.0028	0.0044	!	-	**0.0000**	100s
fc.data	**0.0000**	0.0000	0.0000	0.1729	**0.0000**	0.8141	!	-	**0.0000**	100s
nn_verification	0.0001	0.0756	0.0001	0.1493	0.1493	0.3734	!	-	**0.0000**	100s

this, we applied the spectral clustering algorithm to reclassify the complete datasets, including MIPLib, Coral, ECOGCNN, and Nexp.

Using MIPLib as an example, we employed spectral clustering to partition the dataset into six distinct classes. To evaluate the similarity between these classes, we calculated the average Euclidean distance between instance embeddings from different classes. As shown in the right panel of Figure 4, these distances provide a quantitative measure of class similarity. Furthermore, the intra-class distances, measured using both graph structural and neural encoding metrics, are visualized in Figure 5. The results reveal that the first five classes exhibit relatively small intra-class and inter-class distances, indicating a high degree of similarity among instances. In contrast, the sixth class stands out with significantly larger distances, suggesting the presence of outlier instances.

Based on these findings, we excluded the sixth class from the MIPLib dataset to ensure a more homogeneous classification, thereby improving its suitability for subsequent baseline tests. Detailed classification results for other datasets, including Coral, ECOGCNN, and Nexp, are provided in our online repository.

4.3 Benchmarking Study

To validate the effectiveness of MILPBench and analyze the performance of representative algorithms across different categories, we conducted comparative experiments focusing on objective function values and gap estimation under a consistent wall-clock time limit. This section highlights the performance of three major algorithm categories: classical solvers (Gurobi, SCIP, and CPLEX), heuristic approaches (GLNS, ACP, and RINS), and machine learning (ML)-based methods (Learn2Branch, GNN&GBDT, and Predict&Search). Detailed comparisons within each category are provided in our online repository. Here, we focus on cross-category comparisons and the unique value of MILPBench in evaluating algorithm performance beyond standard benchmarks.

As shown in Table 3, classical solvers exhibit the most robust and reliable performance across diverse problem types. Among them, Gurobi consistently achieves the lowest gap values in real-world problems such as `load_balancing` and `Transportation`, as well as in smaller-scale precision-oriented problems like `MIPlib` and `HEM_knapsack`. CPLEX also demonstrates competitive performance on many instances, particularly in structured problems such as `Cut`. However, the results highlight the limitations of SCIP under time constraints, with significantly higher gap values in complex problems like `SC_hard` and `MVC_hard`. These findings reaffirm the dominance of classical solvers, especially in scenarios requiring optimality and feasibility guarantees.

Heuristic approaches, such as GLNS, ACP, and RINS, demonstrate effectiveness in large-scale problems like `MIS_hard` and `SC_hard`, where computational efficiency is critical. GLNS performs comparably to Gurobi on `MIS_hard`, while RINS achieves reasonable gap values on `MVC_hard` and `Cut`, albeit at the expense of slightly higher computational time. However, heuristic methods struggle in more constrained scenarios, such as `HEM_corlat` and `HEM_mik`, where their inability to handle tight constraints becomes evident.

ML-based methods, including Learn2Branch, GNN&GBDT, and Predict&Search, demonstrate promising results in specific instances but face challenges in broader contexts. Notably, GNN&GBDT achieves strong performance across all IP problems, outperforming all other baselines in these instances. However, as an IP-specific framework, it struggles to generalize to MILP problems, leaving some instances unsolved. Similarly, Predict&Search shows competitive performance on problems like `Cut` and `vary_rhs_s1`, occasionally matching or outperforming heuristic methods. Nevertheless, it also fails to solve certain constrained problems, limiting its overall robustness. Meanwhile, Learn2Branch struggles with large-scale problems like `Coral` and `Cut`, where the exponential growth of the search space significantly impacts its effectiveness.

To visually compare the performance of representative algorithms across diverse problem types, we employed a scoring mechanism based on their gap estimates. Scores were normalized to a 0–100 scale using the Gaussian transformation $\text{Score}(x) = 100 \cdot$

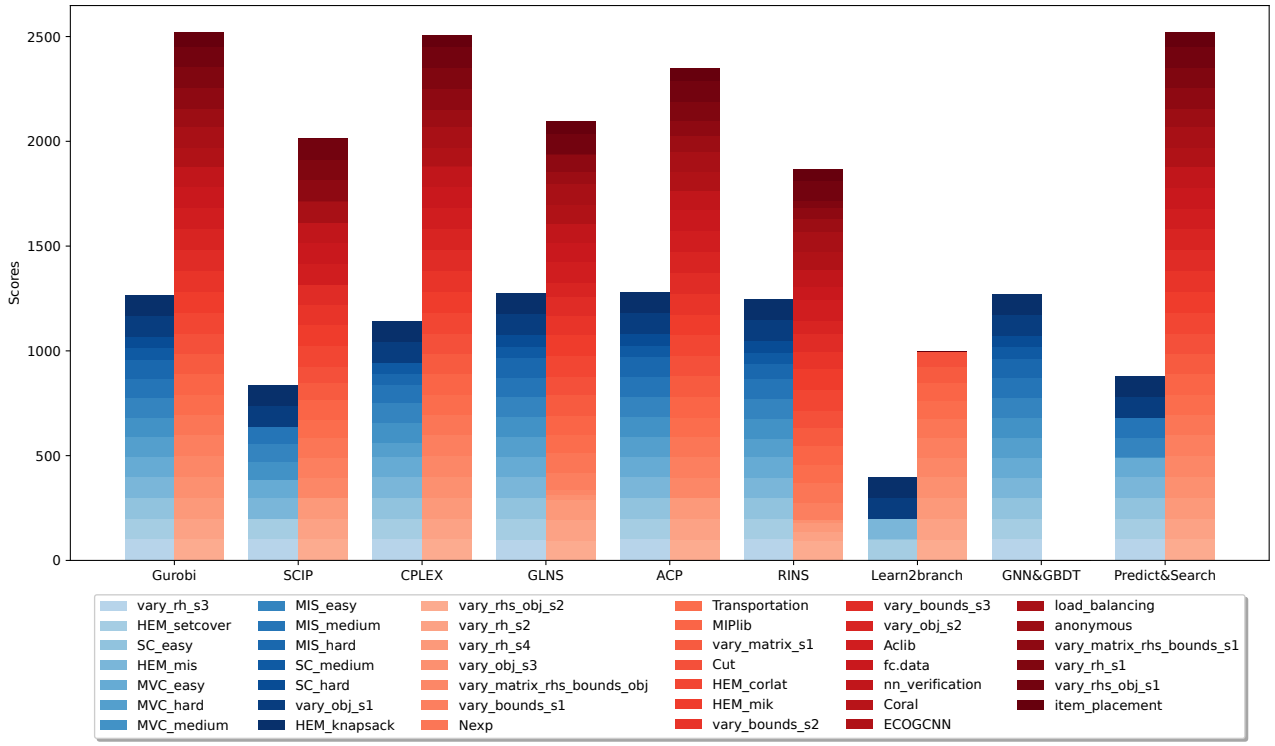


Figure 6: Scores for IP (blue) and MILP (red) problems in MILPBench for different baselines.

$e^{-\frac{x^2}{2}}$, where x is the gap estimate. This approach ensures that algorithms with smaller gap values receive higher scores, providing an objective and standardized metric for cross-category evaluation.

Figure 6 illustrates the normalized scores for integer programming (IP) and mixed-integer linear programming (MILP) problems across all baselines. Classical solvers, particularly Gurobi and CPLEX, consistently achieve higher scores, reflecting their robust performance across both IP and MILP problems. Gurobi excels in precision-oriented and large-scale problems, while CPLEX demonstrates strength in structured instances like `Cut`. In contrast, SCIP achieves relatively lower scores due to its struggles in handling complex and time-constrained problems. Heuristic approaches, such as ACP and RINS, achieve competitive scores on large-scale problems, with ACP consistently outperforming other heuristics. However, their scores decline in constrained scenarios, highlighting the trade-off between computational efficiency and solution robustness. ML-based methods, while showing potential in specific benchmarks, exhibit greater variability. For instance, GNN&GBDT achieves high scores in IP problems but struggles to generalize to MILP instances. Similarly, Predict&Search demonstrates strong performance in structured problems but lacks consistency across other scenarios.

In summary, classical solvers remain the most reliable and versatile algorithms, while heuristics offer computational advantages in large-scale problems. ML-based methods, despite their potential, require further refinement to improve generalizability and scalability. These findings reinforce the importance of MILPBench as a comprehensive benchmarking framework that facilitates detailed performance evaluations and inspires the development of hybrid strategies integrating the strengths of all three categories.

5 Conclusion and Future Work

This paper presented MILPBench, a comprehensive and open-source benchmark dataset for evaluating optimization algorithms in mixed-integer linear programming (MILP). By systematically categorizing problem instances, MILPBench enables structured analyses of algorithm performance, revealing the strengths and limitations of both classical solvers and emerging approaches.

Future work will focus on expanding the dataset to include more diverse MILP problems and enhancing compatibility with evolving optimization methodologies. MILPBench aims to foster rigorous evaluations and drive the development of robust and efficient algorithms, benefiting both research and practical applications.

6 Acknowledgement

This research is supported by the National Research Foundation, Singapore under its AI Singapore Programme (AISG Award No: AISG3-RP-2022-031).

References

[1] David Abramson and Marcus Randall. 1999. A simulated annealing code for general integer linear programs. *Annals of Operations Research* 86, 0 (1999), 3–21.
[2] Tobias Achterberg. 2009. SCIP: solving constraint integer programs. *Mathematical Programming Computation* 1 (2009), 1–41.

[3] Tobias Achterberg and Timo Berthold. 2009. Hybrid branching. In *Integration of AI and OR Techniques in Constraint Programming for Combinatorial Optimization Problems: 6th International Conference, CPAIOR 2009 Pittsburgh, PA, USA, May 27-31, 2009 Proceedings 6*. Springer, 309–311.
[4] Tobias Achterberg, Thorsten Koch, and Alexander Martin. 2005. Branching rules revisited. *Operations Research Letters* 33, 1 (2005), 42–54.
[5] Nathan Adelgren and Akshay Gupte. 2022. Branch-and-bound for biobjective mixed-integer linear programming. *INFORMS Journal on Computing* 34, 2 (2022), 909–933.
[6] Attahiru S Alfa, Bodhaswar T Maharaj, Shruti Lall, and Sougata Pal. 2016. Mixed-integer programming based techniques for resource allocation in underlay cognitive radio networks: A survey. *Journal of Communications and Networks* 18, 5 (2016), 744–761.
[7] Hyung-Chan An, Mohit Singh, and Ola Svensson. 2017. LP-based algorithms for capacitated facility location. *SIAM J. Comput.* 46, 1 (2017), 272–306.
[8] Alper Atamtürk. 2002. On capacitated network design cut–set polyhedra. *Mathematical Programming* 92 (2002), 425–437.
[9] Alper Atamtürk. 2003. On the Facets of the Mixed-Integer Knapsack Polyhedron. *Mathematical Programming* 98, 1-3 (2003), 145–175.
[10] Alper Atamtürk and Juan Carlos Munoz. 2004. A study of the lot-sizing polytope. *Mathematical Programming* 99, 3 (2004), 443–465.
[11] A. Atamtürk and V. Narayanan. 2009. The Submodular 0-1 Knapsack Polytope. 6 (2009), 333–344.
[12] Michel Bénichou, Jean-Michel Gauthier, Paul Girodet, Gerard Hentges, Gerard Ribière, and Olivier Vincent. 1971. Experiments in mixed-integer linear programming. *Mathematical programming* 1 (1971), 76–94.
[13] Livio Bertacco, Matteo Fischetti, and Andrea Lodi. 2007. A feasibility pump heuristic for general mixed-integer problems. *Discrete Optimization* 4, 1 (2007), 63–76.
[14] Timo Berthold. 2006. *Primal heuristics for mixed integer programs*. Ph. D. Dissertation. Zuse Institute Berlin (ZIB).
[15] Suresh Bolusani, Mathieu Besançon, Ambros Gleixner, Timo Berthold, Claudia d'Ambrosio, Gonzalo Muñoz, Joseph Paat, and Dimitri Thomopulos. 2023. The MIP Workshop 2023 Computational Competition on Reoptimization. *arXiv preprint arXiv:2311.14834* (2023).
[16] Alberto Caprara, Paolo Toth, and Matteo Fischetti. 2000. Algorithms for The Set Covering Problem. *Annals of Operations Research* 98 (2000), 353–371.
[17] Li Chen, Hua Xu, Ziteng Wang, Chengming Wang, and Yu Jiang. 2023. Self-paced learning based graph convolutional neural network for mixed integer programming (student abstract). In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 16188–16189.
[18] Ziang Chen, Jialin Liu, Xinshang Wang, Jianfeng Lu, and Wotao Yin. 2022. On representing linear programs by graph neural networks. *arXiv preprint arXiv:2209.12288* (2022).
[19] Ziang Chen, Jialin Liu, Xinshang Wang, Jianfeng Lu, and Wotao Yin. 2023. On Representing Mixed-Integer Linear Programs by Graph Neural Networks. arXiv:2210.10759 [cs.LG] https://arxiv.org/abs/2210.10759
[20] CPLEX User's Manual. 1987. IBM ILOG CPLEX Optimization Studio. *Version* 12, 1987-2018 (1987), 1.
[21] Frank E. Curtis. [n. d.]. COR@L MIP Dataset. https://coral.ise.lehigh.edu/datasets/mixed-integer-instances/.
[22] Emilie Danna, Edward Rothberg, and Claude Le Pape. 2005. Exploring relaxation induced neighborhoods to improve MIP solutions. *Mathematical Programming* 102 (2005), 71–90.
[23] Sven De Vries and Rakesh V Vohra. 2003. Combinatorial auctions: A survey. *INFORMS Journal on computing* 15, 3 (2003), 284–309.
[24] Santanu S Dey, Yatharth Dubey, Marco Molinaro, and Prachi Shah. 2024. A theoretical and computational analysis of full strong-branching. *Mathematical Programming* 205, 1 (2024), 303–336.
[25] Jian-Ya Ding, Chao Zhang, Lei Shen, Shengyin Li, Bing Wang, Yinghui Xu, and Le Song. 2020. Accelerating primal solution findings for mixed integer programs based on solution prediction. In *Proceedings of the aaai conference on artificial intelligence*, Vol. 34. 1452–1459.
[26] Irit Dinur and Samuel Safra. 2005. On The Hardness of Approximating Minimum Vertex Cover. *Annals of mathematics* (2005), 439–485.
[27] Matteo Fischetti, Fred Glover, and Andrea Lodi. 2005. The feasibility pump. *Mathematical Programming* 104 (2005), 91–104.
[28] Christodoulos A Floudas and Xiaoxia Lin. 2005. Mixed integer linear programming in process scheduling: Modeling, algorithms, and applications. *Annals of Operations Research* 139 (2005), 131–162.
[29] Maxime Gasse, Simon Bowly, Quentin Cappart, Jonas Charfreitag, Laurent Charlin, Didier Chételat, Antonia Chmiela, Justin Dumouchelle, Ambros Gleixner, Aleksandr M Kazachkov, et al. 2022. The machine learning for combinatorial optimization competition (ml4co): Results and insights. In *NeurIPS 2021 competitions and demonstrations track*. PMLR, 220–231.
[30] Maxime Gasse, Didier Chételat, Nicola Ferroni, Laurent Charlin, and Andrea Lodi. 2019. Exact combinatorial optimization with graph convolutional neural networks. *Advances in neural information processing systems* 32 (2019).
[31] Sethavidh Gertphol, Yang Yu, Shriram B Gundala, Viktor K Prasanna, Shoukat Ali, Jong-Kook Kim, Anthony A Maciejewski, and Howard Jay Siegel. 2002. A metric and mixed-integer-programming-based approach for resource allocation in dynamic real-time systems. In *Proceedings 16th International Parallel and Distributed Processing Symposium*. IEEE, 10–pp.
[32] Ambros Gleixner, Gregor Hendel, Gerald Gamrath, Tobias Achterberg, Michael Bastubbe, Timo Berthold, Philipp Christophel, Kati Jarck, Thorsten Koch, Jeff Linderoth, et al. 2021. MIPLIB 2017: data-driven compilation of the 6th mixed-integer programming library. *Mathematical Programming Computation* 13, 3 (2021), 443–490.
[33] Chrysanthos E Gounaris, Karthikeyan Rajendran, Ioannis G Kevrekidis, and Christodoulos A Floudas. 2016. Designing networks: A mixed-integer linear optimization approach. *Networks* 68, 4 (2016), 283–301.
[34] Prateek Gupta, Maxime Gasse, Elias Khalil, Pawan Mudigonda, Andrea Lodi, and Yoshua Bengio. 2020. Hybrid models for learning to branch. *Advances in neural information processing systems* 33 (2020), 18087–18097.
[35] Prateek Gupta, Elias B Khalil, Didier Chetélat, Maxime Gasse, Yoshua Bengio, Andrea Lodi, and M Pawan Kumar. 2022. Lookback for learning to branch. *arXiv preprint arXiv:2206.14987* (2022).
[36] LLC Gurobi Optimization. 2021. Gurobi optimizer reference manual. (2021).
[37] Qingyu Han, Linxin Yang, Qian Chen, Xiang Zhou, Dong Zhang, Akang Wang, Ruoyu Sun, and Xiaodong Luo. 2023. A gnn-guided predict-and-search framework for mixed-integer linear programming. *arXiv preprint arXiv:2302.05636* (2023).
[38] He He, Hal Daume III, and Jason M Eisner. 2014. Learning to search in branch and bound algorithms. *Advances in neural information processing systems* 27 (2014).
[39] Jörg Heisterman and Thomas Lengauer. 1991. The efficient solution of integer programs for hierarchical global routing. *IEEE transactions on computer-aided design of integrated circuits and systems* 10, 6 (1991), 748–753.
[40] Gregor Hendel. 2022. Adaptive large neighborhood search for mixed integer programming. *Mathematical Programming Computation* 14, 2 (2022), 185–221.
[41] Mojtaba Heydar, Jie Yu, Yue Liu, and Matthew EH Petering. 2016. Strategic evacuation planning with pedestrian guidance and bus routing: a mixed integer programming model and heuristic solution. *Journal of Advanced Transportation* 50, 7 (2016), 1314–1335.
[42] Weimin Huang, Taoan Huang, Aaron M Ferber, and Bistra Dilkina. 2024. Distributional MIPLIB: a Multi-Domain Library for Advancing ML-Guided MILP Methods. *arXiv preprint arXiv:2406.06954* (2024).
[43] Frank Hutter, Manuel López-Ibánez, Chris Fawcett, Marius Lindauer, Holger H Hoos, Kevin Leyton-Brown, and Thomas Stützle. 2014. AClib: A benchmark library for algorithm configuration. In *Learning and Intelligent Optimization: 8th International Conference, Lion 8, Gainesville, FL, USA, February 16-21, 2014. Revised Selected Papers 8*. Springer, 36–40.
[44] David E Kaufman, Jason Nonis, and Robert L Smith. 1998. A mixed integer linear programming model for dynamic route guidance. *Transportation Research Part B: Methodological* 32, 6 (1998), 431–440.
[45] Thomas N Kipf and Max Welling. 2016. Variational graph auto-encoders. *arXiv preprint arXiv:1611.07308* (2016).
[46] Thorsten Koch, Tobias Achterberg, Erling Andersen, Oliver Bastert, Timo Berthold, Robert E Bixby, Emilie Danna, Gerald Gamrath, Ambros M Gleixner, Stefan Heinz, et al. 2011. MIPLIB 2010: mixed integer programming library version 5. *Mathematical Programming Computation* 3 (2011), 103–163.
[47] Wen-Yang Ku and J Christopher Beck. 2016. Mixed integer programming models for job shop scheduling: A computational analysis. *Computers & Operations Research* 73 (2016), 165–173.
[48] A Land and S Powell. 1979. Computer codes for problems of integer programming. In *Annals of Discrete Mathematics*. Vol. 5. Elsevier, 221–269.
[49] Markus Leitner and Markus Leitner. 2010. *Solving two network design problems by mixed integer programming and hybrid optimization methods*. na.
[50] Kevin Leyton-Brown, Mark Pearson, and Yoav Shoham. 2000. Towards a universal test suite for combinatorial auction algorithms. In *Proceedings of the 2nd ACM conference on Electronic commerce*. 66–76.
[51] Paramet Luathep, Agachai Sumalee, William HK Lam, Zhi-Chun Li, and Hong K Lo. 2011. Global optimization method for mixed transportation network design problem: a mixed-integer linear programming approach. *Transportation Research Part B: Methodological* 45, 5 (2011), 808–827.
[52] Jiaxiang Luo, Jiyin Liu, and Yueming Hu. 2017. An MILP model and a hybrid evolutionary algorithm for integrated operation optimisation of multi-head surface mounting machines in PCB assembly. *International Journal of Production Research* 55, 1 (2017), 145–160.
[53] Vinod Nair, Mohammad Alizadeh, et al. 2020. Neural large neighborhood search. In *Learning Meets Combinatorial Algorithms at NeurIPS2020*.
[54] Vinod Nair, Sergey Bartunov, Felix Gimeno, Ingrid Von Glehn, Pawel Lichocki, Ivan Lobov, Brendan O'Donoghue, Nicolas Sonnerat, Christian Tjandraatmadja, Pengming Wang, et al. 2020. Solving mixed integer programs using neural networks. *arXiv preprint arXiv:2012.13349* (2020).
[55] Napoleão Nepomuceno, Ricardo Saboia, and André Coelho. 2023. A MILP-Based Very Large-Scale Neighborhood Search for the Heterogeneous Vehicle Routing

Problem with Simultaneous Pickup and Delivery. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 330–338.
[56] Kenta Oono and Taiji Suzuki. 2019. Graph neural networks exponentially lose expressive power for node classification. *arXiv preprint arXiv:1905.10947* (2019).
[57] Dimitri J Papageorgiou, George L Nemhauser, Joel Sokol, Myun-Seok Cheon, and Ahmet B Keha. 2014. MIRPLib–A library of maritime inventory routing problem instances: Survey, core model, and benchmark results. *European Journal of Operational Research* 235, 2 (2014), 350–366.
[58] RN Ramlogan and IC Goulter. 1989. Mixed integer model for resource allocation in project management. *Engineering optimization* 15, 2 (1989), 97–111.
[59] Edward Rothberg. 2007. An evolutionary algorithm for polishing mixed integer programming solutions. *INFORMS Journal on Computing* 19, 4 (2007), 534–541.
[60] Franco Scarselli, Marco Gori, Ah Chung Tsoi, Markus Hagenbuchner, and Gabriele Monfardini. 2008. The graph neural network model. *IEEE transactions on neural networks* 20, 1 (2008), 61–80.
[61] Jialin Song, Yisong Yue, Bistra Dilkina, et al. 2020. A general large neighborhood search framework for solving integer linear programs. *Advances in Neural Information Processing Systems* 33 (2020), 20012–20023.
[62] Nicolas Sonnerat, Pengming Wang, Ira Ktena, Sergey Bartunov, and Vinod Nair. 2021. Learning a large neighborhood search algorithm for mixed integer programs. *arXiv preprint arXiv:2107.10201* (2021).
[63] Robert Endre Tarjan and Anthony E Trojanowski. 1977. Finding A Maximum Independent Set. *SIAM J. Comput.* 6, 3 (1977), 537–546.
[64] Mark Turner, Timo Berthold, Mathieu Besançon, and Thorsten Koch. 2023. Branching via cutting plane selection: Improving hybrid branching. *arXiv preprint arXiv:2306.06050* (2023).
[65] Laurence A Wolsey. 2007. Mixed integer programming. *Wiley Encyclopedia of Computer Science and Engineering* (2007), 1–10.
[66] Laurence A Wolsey. 2020. *Integer programming*. John Wiley & Sons.
[67] Yaoxin Wu, Wen Song, Zhiguang Cao, and Jie Zhang. 2021. Learning large neighborhood search policy for integer programming. *Advances in Neural Information Processing Systems* 34 (2021), 30075–30087.
[68] Huigen Ye, Hongyan Wang, Hua Xu, Chengming Wang, and Yu Jiang. 2023. Adaptive constraint partition based optimization framework for large-scale integer linear programming (student abstract). In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 16376–16377.
[69] Huigen Ye, Hua Xu, and Hongyan Wang. 2023. Light-MILPopt: Solving Large-scale Mixed Integer Linear Programs with Small-scale Optimizer and Small Training Dataset. In *The Twelfth International Conference on Learning Representations*.
[70] Huigen Ye, Hua Xu, Hongyan Wang, Chengming Wang, and Yu Jiang. 2023. GNN&GBDT-guided fast optimizing framework for large-scale integer programming. In *International Conference on Machine Learning*. PMLR, 39864–39878.
[71] Ryohei Yokoyama, Yuji Shinano, Syusuke Taniguchi, Masashi Ohkura, and Tetsuya Wakui. 2015. Optimization of energy supply systems by MILP branch and bound method in consideration of hierarchical relationship between design and operation. *Energy conversion and management* 92 (2015), 92–104.
[72] Jiayi Zhang, Chang Liu, Xijun Li, Hui-Ling Zhen, Mingxuan Yuan, Yawen Li, and Junchi Yan. 2023. A survey for solving mixed integer programming via machine learning. *Neurocomputing* 519 (2023), 205–217.
[73] Ziyan Zhao, Shixin Liu, MengChu Zhou, and Abdullah Abusorrah. 2020. Dual-objective mixed integer linear program and memetic algorithm for an industrial group scheduling problem. *IEEE/CAA Journal of Automatica Sinica* 8, 6 (2020), 1199–1209.